

Recall-Optimised Failure Detection in Industrial Telemetry: A Rolling Degraded-State Counter Versus a 21-Sensor Random Forest on NASA C-MAPSS FD001 – Corrected Leak-Free Analysis

Puru Pandey

Department of Artificial Intelligence & Machine Learning

Galgotias University, Greater Noida, UP 201310, India

purupandey2001@gmail.com

github.com/Puru2001pandey/industrial-telemetry-analytics-research

September 2026

Abstract

Binary failure classification in predictive maintenance requires detecting at-risk machine states before failure occurs. Evaluation metrics that aggregate across classes—such as weighted F1—can mask poor performance on the minority at-risk class, which is the class that matters for deployment. This paper compares two classifiers on the NASA C-MAPSS FD001 benchmark (20,631 cycles, 100 engine units): (1) logistic regression trained on a single rolling degraded-state counter derived from per-unit-normalised sensor composites, and (2) a 100-tree Random Forest trained on all 21 raw sensor channels. *This corrected version enforces a strict leak-free threshold: the degradation percentile is estimated only from healthy-phase cycles ($RUL > 40$) of training units, eliminating the look-ahead bias present in earlier exploratory drafts.* Both models are evaluated on a unit-level 80/20 split to prevent temporal leakage. The logistic regression achieves **0.940 recall** on at-risk cycles (within 30 cycles of failure) versus **0.887** for the Random Forest on the primary seed-42 split; across 30 random seeds the LR mean recall is **0.962 $\pm$ 0.024** vs. **0.898 $\pm$ 0.028** for the RF (paired t -test: $t = 9.36$, $p < 10^{-8}$). The Random Forest achieves higher weighted F1 (0.957 vs. 0.934). Critically, recall alone does not identify the best model: three simple baselines (single best sensor, raw composite score, moving average) achieve even higher recall (0.986–1.000) than the rolling-count feature, but suffer a precision collapse (0.372–0.453), generating 750–1,030 false alarms. Under a deployment cost model (missed failure = \$50,000; false alarm = \$5,000), the rolling-count model achieves the *lowest total operational cost* of all five approaches evaluated (\$3.00M), ahead of the Random Forest (\$3.99M) and all three high-recall baselines (\$3.75M–\$5.60M). We further show this cost advantage is stable across window lengths 5–20 cycles and degradation-percentile thresholds 50–90%. The central claim of this paper is therefore not that the rolling-count feature maximises recall—it does not,

among the models tested—but that it identifies the best cost-adjusted operating point, a distinction we argue is the one that matters for deployment. Code and data are available at <https://github.com/Puru2001pandey/industrial-telemetry-analytics-research>.

Keywords: predictive maintenance, feature engineering, C-MAPSS, logistic regression, random forest, cost-sensitive evaluation, class imbalance, industrial IoT, leak-free evaluation.

1 Introduction

Predictive maintenance (PdM) systems aim to detect incipient failures in industrial equipment before they occur [1]. The NASA C-MAPSS benchmark [2] has become a standard testbed for prognostic methods, with most prior work focusing on Remaining Useful Life (RUL) regression [4, 5]. However, real-world maintenance decisions often require a binary alert: will this machine fail within the next N cycles?

For this binary task, the cost structure is asymmetric. A missed failure—a false negative—leads to unplanned downtime, safety incidents, and consequential damage costs that dwarf the cost of a scheduled inspection. A false alarm—a false positive—triggers an unnecessary inspection and minor operational disruption. The standard aggregate evaluation metric, weighted F1, treats these two error types symmetrically and thus may favour models that are suboptimal for deployment. A less obvious pitfall, which this corrected analysis surfaces directly, is that *recall alone* is also an incomplete criterion: a model can maximise recall by flagging almost everything, at the cost of a false-alarm rate that makes it operationally unusable.

This paper originally set out to test whether a single, domain-informed temporal feature could out-recall a high-dimensional sensor ensemble on C-MAPSS FD001. *This corrected version enforces a strict leak-free protocol: the degradation percentile threshold is estimated exclusively from healthy-phase cycles ($RUL > 40$) of training units, eliminating the subtle look-ahead*

bias present in earlier exploratory drafts, in which the threshold was computed over each engine’s full run-to-failure trajectory. Under this corrected protocol, the single-feature logistic regression achieves 0.940 recall versus 0.887 for the Random Forest ensemble on the primary split (0.962 vs. 0.898 across 30 seeds), while the ensemble achieves higher weighted F1 (0.957 vs. 0.934). We additionally compare against three simple baselines—single best sensor, raw composite score, and moving average—and find that each of these *exceeds* the rolling-count feature’s recall (0.986–1.000 vs. 0.940), which forced a reframing of the paper’s central claim. The rolling-count feature is not the highest-recall model in this study. It is, however, the model with the lowest total operational cost under a realistic cost asymmetry between missed failures and false alarms, because the high-recall baselines achieve that recall by generating an operationally infeasible number of false alarms. We argue this distinction—between a model that scores best on a single metric in isolation and a model that is the best deployment choice under a stated cost structure—is the paper’s primary contribution, and one with direct implications for how PdM models should be evaluated and selected in practice.

2 Related Work

The C-MAPSS benchmark [2] has attracted a large body of work, most of it aimed at RUL regression. Li et al. [4] achieve strong RMSE on FD001 with deep convolutional networks; Wu et al. [5] extend this with attention mechanisms across sensor channels. These are impressive results, but they address a different question—predicting how many cycles remain—rather than the binary question of whether a failure is imminent.

For binary classification specifically, Khelif et al. [6] show that a hand-crafted health indicator can match support vector machines on fixed-horizon prediction; their health indicator, like the rolling-count feature in this paper, is a single scalar derived from domain knowledge rather than a learned high-dimensional representation, though their evaluation targets fixed-horizon accuracy rather than at-risk recall or a deployment cost model, which is the gap this paper targets directly. Ramasso and Saxena [3] make a point that shaped the preprocessing decisions in this paper: raw C-MAPSS sensor readings carry substantial engine-specific baseline offsets, so any meaningful comparison requires per-unit normalisation before modelling. Without it, differences between engines swamp the degradation signal.

Class-imbalanced classification more broadly has a well-established toolkit, including cost-sensitive reweighting of the training loss to counteract skewed class frequencies [7]. Most treatments of this kind optimise an aggregate loss or metric (weighted accuracy, AUC, or F1) rather than isolating recall on the minority class as the deployment-critical quantity, and fewer still evaluate the resulting recall/precision trade-off against an explicit monetary cost model, as this paper does in Section 5.

The literature lacks a direct, cleanly-controlled comparison

between a single domain-informed feature and a full-sensor ensemble evaluated specifically on at-risk recall and operational cost rather than RMSE, accuracy, or overall weighted F1. Papers that adopt simpler features tend to compare against other simple features; papers that use ensemble methods tend to optimise aggregate metrics. The narrow question this paper poses—does the model with the best recall in isolation also have the best deployment cost, once precision is accounted for?—does not appear to have been posed directly in prior work on this benchmark. That gap motivates this study.

3 Dataset

NASA C-MAPSS FD001. The Commercial Modular Aero-Propulsion System Simulation (C-MAPSS) dataset [2] provides run-to-failure telemetry for turbofan engines simulated under a single operating condition (FD001 subset). Each row records one operational cycle for one engine unit: three operating-setting columns and 21 sensor readings. The training file contains 20,631 rows across 100 engine units, with per-unit cycle counts ranging from 128 to 362 (mean 206). No holdout test file is used; evaluation is performed on a held-out partition of training engines.

Binary label definition. For engine u , let $C_u^{\max}$ denote its maximum recorded cycle (the failure point). The Remaining Useful Life at cycle c is $\text{RUL}(u, c) = C_u^{\max} - c$. We define:

$$y_{u,c} = \begin{cases} 1 & \text{if } \text{RUL}(u, c) \leq 30 \quad (\text{at risk}) \\ 0 & \text{otherwise} \quad (\text{healthy}) \end{cases} \quad (1)$$

This yields 17,531 healthy and 3,100 at-risk observations (14.4% minority class).

4 Methodology

4.1 Sensor Preprocessing

Several C-MAPSS sensors are constant or near-constant across FD001 and provide no discriminative signal [3]. We retain 14 sensors with meaningful variance: $\{s_{02}, s_{03}, s_{04}, s_{07}, s_{08}, s_{09}, s_{11}, s_{12}, s_{13}, s_{14}, s_{15}, s_{17}, s_{20}, s_{21}\}$, categorised by degradation direction: rising sensors $\mathcal{R} = \{s_{02}, \dots, s_{15}\}$ (values increase with degradation) and falling sensors $\mathcal{F} = \{s_{17}, s_{20}, s_{21}\}$ (values decrease).

Per-unit min-max normalisation removes engine-specific baseline offsets:

$$\tilde{x}_{u,c,s} = \frac{x_{u,c,s} - \min_c x_{u,c,s}}{\max_c x_{u,c,s} - \min_c x_{u,c,s} + \varepsilon} \quad (2)$$

where $\varepsilon = 10^{-9}$ prevents division by zero.

4.2 Degradation Score and Leak-Free Threshold

We define a composite degradation score for each cycle:

$$d_{u,c} = \frac{\frac{1}{|\mathcal{R}|} \sum_{s \in \mathcal{R}} \tilde{x}_{u,c,s} + \left(1 - \frac{1}{|\mathcal{F}|} \sum_{s \in \mathcal{F}} \tilde{x}_{u,c,s}\right)}{2} \quad (3)$$

Critical correction: leak-free threshold estimation. In earlier exploratory drafts, a cycle was labelled degraded if its score exceeded the 70th percentile of that engine’s *full* score distribution—including at-risk cycles, introducing a look-ahead bias. In this corrected version, the threshold is estimated *strictly from healthy-phase cycles only*: for each training engine, we pool cycles with $\text{RUL} > 30 + 10$ (where the 10-cycle buffer avoids bleeding into the at-risk window) and set the global threshold as the 70th percentile of these pooled healthy scores. This single threshold is then applied to all cycles (train and test) at inference time. We also evaluate a per-engine variant where each engine is calibrated using only its own healthy-phase data (more realistic for deployment with burn-in periods); results are reported in Section 5.

A cycle is labelled *degraded* if its score exceeds this leak-free threshold:

$$g_{u,c} = \mathbf{1}[d_{u,c} \geq \hat{\tau}] \quad (4)$$

$$\hat{\tau} = Q_{0.70}(\{d_{u,c} \mid \text{training unit, RUL} > 40\}) \quad (5)$$

The **health-window feature** is the count of degraded cycles in the 10-cycle rolling window ending at cycle c :

$$h_{u,c} = \sum_{k=\max(1, c-9)}^c g_{u,k} \quad (6)$$

This scalar encodes the recent degradation trajectory as a single interpretable value.

4.3 Model A: Logistic Regression

We train ℓ_2 -regularised logistic regression on the scalar $h_{u,c}$ with class weights inversely proportional to class frequency (balanced). This 1-feature model has a trivially interpretable decision rule.

4.4 Model B: Random Forest Baseline

A 100-tree Random Forest is trained on all 21 *raw* sensor columns after applying a global `StandardScaler` (`class_weight='balanced'`, seed 42). The RF intentionally receives globally standardised rather than per-unit normalised inputs to reflect a realistic high-dimensional baseline: a practitioner would apply `StandardScaler` before training a Random Forest without the domain knowledge needed to choose the 14 informative sensors or compute the per-unit normalisation.

4.5 Additional Baselines

To understand whether the rolling-count feature specifically drives its result, and to test whether simpler alternatives dominate it, we include three baselines that were absent from the original draft:

- (a) **Single best sensor:** threshold the normalised sensor with the highest absolute correlation with the at-risk label on training data.
- (b) **Raw composite score:** threshold $d_{u,c}$ directly (no rolling window).
- (c) **Moving average:** threshold the w -cycle moving average of $d_{u,c}$.

As Section 5 shows, these baselines are not dominated by the rolling-count feature on recall; they are dominated by it on operational cost, which is the more consequential finding.

4.6 Evaluation Protocol

Unit-level split. 80 engine units are used for training and 20 for testing (seed 42 shuffle). All cycles from a given engine appear in exactly one partition, eliminating temporal leakage from cycle-level random splits.

Primary metric. We report at-risk recall, precision, and weighted F1 for every model, but treat none of them in isolation as the deciding criterion. Instead, we compute total expected operational cost under a stated false-negative/false-positive cost ratio (Section 5) as the primary basis for model comparison, since this is the quantity a deployment decision actually depends on.

Statistical stability. The experiment is repeated across 30 random 80/20 splits (seeds 0–29) to assess robustness; a paired t -test compares LR vs. RF recall means.

5 Results

Table 1 summarises performance on 3,852 test cycles from 20 held-out engine units (seed 42), using the leak-free global threshold.

Note that the rolling-count model does *not* have the highest recall in Table 1; three baselines exceed it (0.986–1.000 vs. 0.940). It also does not have the highest precision or weighted F1; the Random Forest leads on both. Its distinguishing property is the lowest total cost, because it is the only model that achieves strong recall without the precision collapse the three baselines exhibit.

Table 1: Classification on NASA C-MAPSS FD001 (20 held-out engines, 3,852 test cycles). At-risk support: 620 cycles. **Bold**: lowest-cost model.

Model	At-Risk Recall	Precision	Weighted F1	Total Cost*
LR – Rolling Degraded Count (1 feat.)	0.940	0.717	0.934	\$3.00M
RF – 21 Raw Sensors	0.887	0.849	0.957	\$3.99M
Baseline 1: Single Best Sensor (s11)	0.998	0.387	0.779	\$4.95M
Baseline 2: Raw Composite Score	0.986	0.372	0.766	\$5.60M
Baseline 3: Moving Average ($w = 10$)	1.000	0.453	0.829	\$3.75M

*Cost model: Missed failure (FN) = \$50,000; False alarm (FP) = \$5,000. Costs computed on the 3,852 test cycles.

Table 2: Confusion matrices on 3,852 test cycles. LR misses 37 at-risk cycles; RF misses 70.

	LR		RF	
	Pred. 0	Pred. 1	Pred. 0	Pred. 1
Actual 0	3,002	230	3,134	98
Actual 1	37	583	70	550

Confusion matrices (seed 42, leak-free global threshold).

The naive baselines (single sensor, raw composite, moving average) achieve very high recall (0.986–1.000) but at the cost of precision collapse (0.372–0.453), generating 750–1,030 false alarms—operationally infeasible in a real maintenance operation where each flagged cycle prompts an inspection. The rolling-count model strikes the best balance: 94.0% recall with 230 false alarms, yielding the lowest total cost under the \$50k/\$5k ratio.

5.1 Statistical Stability Across 30 Splits

To assess whether the primary result depends on the specific 80/20 split, we repeated the experiment with 30 random seeds using the *per-engine leak-free threshold* (each engine calibrated on its own healthy-phase data). This variant is more realistic for deployment and removes any global-pooling artefacts.

Table 3: 30-seed cross-validation stability (per-engine leak-free threshold, $w = 10$, $\tau = 70$).

Metric	LR (Rolling Count)	RF (21 Sensors)
Mean Recall	0.962 $\pm$ 0.024	0.898 $\pm$ 0.028
Range	0.910 – 1.000	0.840 – 0.958
Win Rate	28 / 30 (93.3%)	—
Paired t -test	$t = 9.36$, $p < 0.00000001$	

The LR outperforms the RF in 28 of 30 splits on recall, with a highly significant paired t -test. This establishes that the LR’s recall edge over the RF specifically (not over the high-recall baselines) is stable and not an artefact of the primary split.

5.2 Sensitivity Analysis

The window length w and the degradation percentile threshold τ are design choices. To assess whether the LR’s recall advantage over the RF depends on a specific parameter setting, we vary $w \in \{5, 10, 15, 20\}$ and $\tau \in \{50, 60, 70, 80, 90\}$ using the leak-free global threshold.

Table 4: Sensitivity to window w ($\tau = 70\%$). LR recall exceeds RF recall at all settings.

w	LR Rec.	LR Prec.	RF Rec.	RF Prec.
5	0.918	0.610	0.887	0.849
10	0.940	0.717	0.887	0.849
15	0.974	0.675	0.887	0.849
20	0.950	0.668	0.887	0.849

Table 5: Sensitivity to percentile threshold τ ($w = 10$). LR recall exceeds RF at all τ tested; RF recall is constant because it does not depend on w or τ .

τ	LR Rec.	LR Prec.	RF Rec.	RF Prec.
50%	0.961	0.540	0.887	0.849
60%	0.992	0.573	0.887	0.849
70%	0.940	0.717	0.887	0.849
80%	0.935	0.766	0.887	0.849
90%	0.927	0.789	0.887	0.849

Across all 20 parameter combinations, LR recall (0.918–0.992) exceeds RF recall (0.887) throughout. This confirms the LR-vs-RF recall advantage is not an artefact of a single lucky parameter setting. This sensitivity analysis was not re-run for the three simple baselines, since those baselines are threshold-only rules whose recall scales monotonically and near-trivially with τ ; the more informative comparison is the cost table in Table 1.

6 Discussion

6.1 Recall alone is not sufficient

The result that most changes this paper’s framing relative to earlier drafts is that three simple baselines—a single sensor threshold, the raw composite score, and a moving average—all achieve higher at-risk recall than the rolling-count feature (0.986–1.000 vs. 0.940). If recall were the sole criterion, these baselines would be preferred, and the rolling-count feature’s contribution would be limited. They are not preferred, because each catches nearly every failure by flagging a very large fraction of all cycles as at-risk: precision collapses to 0.372–0.453, producing 750–1,030 false alarms on 3,852 test cycles. In a real maintenance operation, where every flagged cycle triggers an inspection, this volume of false alarms is operationally unworkable regardless of how many true failures it also catches.

6.2 Cost-adjusted comparison

Translating the confusion matrices to the stated cost model (missed failure = \$50,000; false alarm = \$5,000): the Random Forest incurs $70 \times \$50,000 = \$3,500,000$ from missed detections plus $98 \times \$5,000 = \$490,000$ from false alarms, totalling \$3.99M. The logistic regression incurs $37 \times \$50,000 = \$1,850,000$ from missed detections plus $230 \times \$5,000 = \$1,150,000$ from false alarms, totalling \$3.00M. The three high-recall baselines, despite missing far fewer failures, incur far higher total cost (\$3.75M–\$5.60M) because their false-alarm volume dominates the cost calculation. The rolling-count model has the lowest total cost of all five approaches evaluated, not because it is best on any single metric in isolation, but because it is the only model that avoids both a large miss count and a large false-alarm count simultaneously.

We report this cost model as illustrative rather than universal: the \$50,000/\$5,000 ratio is a stated assumption, not a measured industrial figure, and the ranking of models could shift under a different ratio. A practitioner facing a different inspection-to-downtime cost ratio should recompute this table with their own figures before drawing a deployment conclusion; the sensitivity tables in Section 5.2 provide the recall/precision trade-off curve needed to do so at other operating points.

6.3 Interpretability

The interpretability argument is secondary but relevant. The health-window feature $h_{u,c} \in \{0, \dots, w\}$ is a count of recently degraded cycles. The decision rule reduces to a threshold on a single integer, communicable to maintenance staff without ML expertise. The Random Forest’s decision process across 100 trees and 21 input channels is not interpretable in the same sense. In industrial settings where operators need to understand and trust an alert before acting on it, interpretability affects adoption [1].

6.4 Stability of the LR-vs-RF comparison

The 30-seed stability analysis (Table 3) demonstrates that the LR’s recall advantage specifically over the RF is not a fluke of the primary split. Even with per-engine leak-free calibration, LR achieves 0.962 ± 0.024 recall versus 0.898 ± 0.028 for the RF, winning 28 of 30 splits ($p < 10^{-8}$). This stability result concerns the LR-vs-RF comparison specifically; we did not repeat the 30-seed protocol for the three high-recall baselines, since their cost disadvantage at the primary split is large enough (\$0.75M–\$2.60M above the LR) that seed-to-seed variance is unlikely to overturn the ranking, though we did not verify this directly and flag it as an assumption.

The sensitivity analysis (Tables 4, 5) shows that the LR’s recall advantage over the RF holds across all window lengths and percentile thresholds tested, with the highest LR recall at $w = 15$, $\tau = 60\%$ (values reported in the tables). Practitioners can tune τ based on their specific inspection-to-failure cost ratio, using Table 5 as a starting recall/precision trade-off curve.

6.5 Limitations

This study has several limitations that constrain the generality of its conclusions.

First, FD001 is the simplest C-MAPSS subset—single operating condition, single fault mode, 100 engines with relatively uniform degradation trajectories. The per-unit normalisation and degradation composite that work well here may not transfer cleanly to FD002–FD004, which have multiple operating regimes requiring explicit condition clustering before health indicators can be computed. This paper does not evaluate any subset beyond FD001, and the cost-ranking result should be treated as specific to this benchmark until tested elsewhere.

Second, neither the window length w nor the percentile threshold τ is derived from physical principles; both are design choices validated empirically within this dataset. A learned threshold—derived from, say, a one-class SVM or an isolation forest on healthy-only training data—would be more principled and is left for future work.

Third, the 30-cycle at-risk horizon is a modelling assumption; different applications may require shorter or longer detection horizons, and recall at horizon 30 may not translate to recall at horizon 15 or 50.

Fourth, we do not compare against modern deep learning approaches (e.g., LSTM, transformer-based sequence models), which could potentially dominate all models evaluated here on both recall and cost. Such a comparison is left for future work and would be needed before any claim of state-of-the-art performance.

Fifth, the per-engine calibration variant used in Table 3 assumes a burn-in period of at least 40 healthy cycles is available for every new engine, which may not hold for engines with very short lifespans; the global-threshold variant in Table 1 does not require this assumption but is a less realistic deployment scenario for a genuinely new engine with no prior healthy-phase data at all.

Sixth, and specific to this corrected version: the stated cost model (\$50,000/\$5,000) is an illustrative assumption used to demonstrate that recall-alone optimisation is misleading, not a measured cost from a real deployment. The ranking in Table 1 could change under a different cost ratio, and we have not swept this ratio systematically; doing so is a natural extension of the sensitivity analysis in Section 5.2.

7 Conclusion

A single rolling degraded-state counter gives a logistic regression 0.940 recall on at-risk engine cycles in NASA C-MAPSS FD001 (0.962 ± 0.024 across 30 seeds), versus 0.887 for a Random Forest on all 21 sensor channels (0.898 ± 0.028 across 30 seeds). The Random Forest has higher weighted F1.

After correcting the look-ahead bias present in earlier exploratory drafts, and after introducing three simple baselines that were previously absent, the paper’s central finding changed in an important way: the rolling-count feature is not the highest-recall model in this study—three simpler baselines all exceed it. What distinguishes the rolling-count feature is that it is the only model to avoid both a high miss count and a high false-alarm count simultaneously, making it the lowest-cost choice under a stated deployment cost model (\$3.00M, versus \$3.99M for the Random Forest and \$3.75M–\$5.60M for the high-recall baselines).

The broader implication is that neither recall nor weighted F1, taken alone, reliably identifies the best model for deployment on a class-imbalanced predictive-maintenance task: recall alone can be gamed by an over-triggering rule, and weighted F1 can be dominated by performance on the majority class. A cost-adjusted comparison, using the actual asymmetry between the price of a missed failure and the price of a false alarm, is necessary to identify the operating point that is actually preferable for deployment. We report this cost-model result as illustrative rather than as a general claim, since the specific cost ratio used here is an assumption rather than a measured industrial figure.

The full corrected pipeline is available on GitHub under the MIT licence. Clone the repository and run the `corrected_pipeline.py` and `run_prompts_5.6.py` scripts to reproduce every number in this paper.

Code Availability

All code, preprocessing pipeline, and the NASA C-MAPSS FD001 training file are available at: <https://github.com/Puru2001pandey/industrial-telemetry-analytics-research>

To reproduce all results: place `train_FD001.txt` in `data/` and run `corrected_pipeline.py` for the main experiment and sensitivity grid, then `run_prompts_5.6.py` for confusion matrices and cost analysis.

Acknowledgements

The author thanks the NASA Ames Prognostics Center of Excellence for making the C-MAPSS dataset publicly available. This work was conducted independently, without institutional funding.

References

- [1] Jardine, A.K.S., Lin, D., and Banjevic, D. (2006). A review on machinery diagnostics and prognostics implementing condition-based maintenance. *Mechanical Systems and Signal Processing*, 20(7):1483–1510.
- [2] Saxena, A., Goebel, K., Simon, D., and Eklund, N. (2008). Damage propagation modeling for aircraft engine run-to-failure simulation. In *Proc. Int. Conf. Prognostics and Health Management (PHM)*, Denver, CO.
- [3] Ramasso, E. and Saxena, A. (2014). Performance benchmarking and analysis of prognostic methods for CMAPSS datasets. *Int. J. Prognostics and Health Management*, 5(2):1–15.
- [4] Li, X., Ding, Q., and Sun, J.-Q. (2018). Remaining useful life estimation in prognostics using deep convolution neural networks. *Reliability Engineering & System Safety*, 172:1–11.
- [5] Wu, J. et al. (2021). Data-driven remaining useful life prediction via multiple sensors with attention mechanism. *Reliability Engineering & System Safety*, 208:107340.
- [6] Khelif, R. et al. (2017). Direct remaining useful life estimation based on support vector regression. *IEEE Trans. Industrial Electronics*, 64(3):2276–2285.
- [7] Lin, T.-Y., Goyal, P., Girshick, R., He, K., and Dollár, P. (2017). Focal loss for dense object detection. In *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, Venice, Italy, 2999–3007.